Contratos de datos como mecanismo de detección temprana de errores en arquitecturas de datos por capas: una evaluación cuantitativa del compromiso entre costo de verificación y costo de reprocesamiento

Leandro Ismael Rojas
Legajo 0140050 — Licenciatura en Informática
Universidad de Palermo, Buenos Aires, Argentina

Gonzalo Javier Díaz
Tutor — Trabajo Final de Grado
Universidad de Palermo

***Abstract*—Los procesos analíticos organizados en capas propagan hacia adelante los defectos que reciben del origen, de modo que un dato inválido suele hacerse visible recién cuando alguien advierte una inconsistencia en un resultado. Los contratos de datos se proponen en la industria como mecanismo de detección temprana, pero la decisión de adoptarlos se apoya en argumentos cualitativos: verificar cuesta en cada ejecución, mientras que el ahorro se materializa solo cuando el defecto ocurre. Este trabajo mide ese compromiso mediante un experimento controlado sobre un conjunto de datos público. Un proceso de tres capas con modelo dimensional se ejecuta en tres variantes —sin verificación, con las restricciones nativas del motor de almacenamiento, y con contratos verificados en cada frontera— y se le inyectan seis clases de falla derivadas de dimensiones normalizadas de calidad, en 105 corridas instrumentadas. Los contratos detectan la totalidad de las clases inyectadas y las restricciones del motor solo una parte: quedan ciegas a las semánticas. La respuesta al compromiso de costo no es un valor único sino una curva de punto de equilibrio, que ubica la frecuencia de defectos a partir de la cual verificar sale más barato que reprocesar.**

***Index Terms*—arquitectura por capas, calidad de datos, contratos de datos, detección temprana, integridad referencial, inyección de fallas, lakehouse, ODCS**

I. Introducción

Los repositorios analíticos modernos organizan el procesamiento de datos en capas sucesivas de refinamiento: una capa de ingesta que conserva el dato tal como llega del origen, una capa intermedia que lo limpia y lo modela, y una capa final orientada al consumo por reportes y tableros. Cada capa se alimenta de la anterior, de modo que un defecto introducido en la ingesta se propaga hacia adelante y suele hacerse visible recién cuando un usuario advierte una inconsistencia en un resultado. Para entonces, la corrección exige recalcular el proceso completo y el dato defectuoso ya estuvo disponible para la toma de decisiones.

El problema de fondo es organizacional además de técnico. Cuando el equipo que produce un dato y el que lo consume son distintos, el origen puede cambiar en cualquier momento sin aviso, y aun los cambios concebidos como mejoras pueden producir efectos perjudiciales difíciles de diagnosticar en los consumidores [1]. Un sistema de validación desplegado a gran escala en la industria documenta fallas de este tipo: datos que llegan con la estructura esperada pero con contenido inválido, y que atraviesan el proceso sin generar error alguno [2]. Una encuesta industrial a doscientos profesionales de datos —literatura gris, sujeta por lo tanto a las reservas metodológicas del caso [3]— aporta un indicio del alcance que tiene el fenómeno: tres de cada cuatro participantes informan que, siempre o casi siempre, quien advierte primero el defecto es el usuario de negocio que consume el dato [4].

Como respuesta, la industria propuso los *contratos de datos*, interfaces acordadas entre quien genera un dato y quien lo consume [5]. En su forma estandarizada son especificaciones declarativas y versionadas del esquema, los tipos, las restricciones y las expectativas de calidad que una fuente se compromete a cumplir, expresadas en un formato legible por máquina que permite verificar automáticamente cada carga contra lo pactado [6]. Sin embargo, la adopción del concepto avanzó desde la práctica profesional con escaso acompañamiento académico [7], y la decisión de incorporar esta verificación se funda habitualmente en argumentos cualitativos: verificar introduce un costo en cada ejecución, mientras que el ahorro se materializa únicamente cuando el defecto ocurre. No se dispone de mediciones publicadas de ese compromiso en arquitecturas analíticas por capas.

Este trabajo aborda ese vacío mediante un experimento controlado. Se construye un proceso de tres capas con modelo dimensional sobre un conjunto de datos público, se lo ejecuta en tres variantes —sin verificación, con las restricciones nativas del motor de almacenamiento, y con contratos de datos verificados en cada frontera— y se inyecta un conjunto controlado y repetible de seis clases de falla derivadas de dimensiones normalizadas de calidad de datos. Se mide qué proporción de defectos detecta cada variante, en qué capa, qué volumen defectuoso alcanza el consumo y a qué costo, para determinar bajo qué condiciones la adopción de contratos

resulta justificada.

El resto del trabajo se organiza de la siguiente manera. La Sección II presenta los conceptos de calidad de datos, arquitecturas por capas y contratos de datos; la Sección III reseña los antecedentes; la Sección IV describe el diseño experimental; la Sección V informa las mediciones, y la Sección VI las interpreta a la luz de las hipótesis. Cierran el trabajo las conclusiones y las líneas futuras.

II. Marco teórico

II-A. Calidad de datos y sus dimensiones

La calidad de datos se define como el grado en que las características de los datos satisfacen necesidades explícitas e implícitas bajo condiciones especificadas. La norma ISO/IEC 25012 [8] establece un modelo de quince características consideradas desde dos puntos de vista: el *inherente*, que refiere al dato en sí mismo, y el *dependiente del sistema*, que refiere a cómo el sistema lo preserva. Cinco características son exclusivamente inherentes —exactitud, completitud, consistencia, credibilidad y actualidad—, tres son exclusivamente dependientes del sistema y las siete restantes participan de ambos puntos de vista. Esta clasificación tiene antecedentes en la literatura de sistemas de información, que derivó las dimensiones de calidad desde la perspectiva del consumidor del dato [9]; esa misma literatura sistematizó después las metodologías para evaluarlas y mejorarlas [10]. Las seis clases de falla estudiadas en este trabajo (Sección IV) se corresponden con características del grupo inherente, lo cual es esperable: son defectos del dato, no del sistema que lo almacena.

Tres de esas características alcanzan para describir las fallas estudiadas. La *exactitud* es el grado en que el dato representa correctamente el valor verdadero del atributo, y, en su aspecto sintáctico, la cercanía de los valores al conjunto definido como válido en su dominio; la *completitud* es el grado en que el dato tiene valores para todos los atributos esperados, y la *consistencia*, el grado en que está libre de contradicciones y es coherente con otros datos [8]. La literatura antecedente agrupa las dimensiones de otra manera —en categorías intrínseca, contextual, de representación y de accesibilidad— y reserva la intrínseca para la calidad que el dato tiene por sí mismo [9].

II-B. Arquitecturas analíticas por capas

El patrón de refinamiento progresivo tiene raíces en las áreas de *staging* del *data warehouse* clásico y evolucionó, con la aparición del *lakehouse* [11], [12], hacia el esquema de tres capas popularizado por la industria bajo el nombre de arquitectura *medallion* [13]. Cabe señalar que este término proviene de documentación de proveedores y no de literatura revisada por pares; su uso en este trabajo se apoya en el reconocimiento de la literatura gris como fuente del estado de la práctica, con los recaudos de calidad que proponen las guías para su incorporación en investigación de ingeniería de software [3]. El formato de tabla empleado, Delta Lake, aporta transacciones ACID (atomicidad, consistencia, aislamiento y durabilidad) sobre almacenamiento de objetos y un registro de transacciones que deja constancia de cada operación [14] y que, para las escrituras, informa métricas de volumen [15]; ese registro se emplea aquí como instrumento de medición externo al código desarrollado.

En el esquema de tres capas, la primera conserva el dato tal como llega, la segunda lo valida y la tercera lo enriquece para el consumo; la denominación bronce, plata y oro alude a la calidad del dato en cada una [13]. La capa de ingesta cumple el papel del área de *staging* que recibía los datos antes del almacén en el *data warehouse* clásico [11]. Este trabajo usa las denominaciones ingesta, intermedia y consumo, que no dependen de un proveedor. Sobre las tablas, Delta Lake admite dos tipos de restricción: `NOT NULL`, que impide valores nulos en una columna, y `CHECK`, que exige que una expresión booleana sea verdadera en cada fila. Cuando una restricción se viola, el motor lanza una excepción e impide que los datos nuevos se agreguen; la de nulabilidad se declara en el esquema al crear la tabla [16].

II-C. Contratos de datos

El concepto de contrato aplicado al intercambio de datos registra antecedentes académicos tempranos en el contexto de mercados de datos en la nube [17], y resurgió en la última década desde la práctica industrial, asociado al principio de tratar los datos como producto. Una revisión sistemática reciente de la literatura gris [7] precisa la definición, identifica las motivaciones de adopción y propone una agenda de investigación, y constata la escasez de estudios académicos sobre el tema. Este trabajo emplea el *Open Data Contract Standard* (ODCS) [6], especificación abierta mantenida bajo la Linux Foundation AI & Data que integra esquema, tipos, restricciones y reglas de calidad en un único documento YAML versionado.

Un contrato ODCS se organiza en secciones: fundamentos, esquema, referencias, calidad de datos, canales de soporte, precios, equipo, roles, acuerdo de nivel de servicio, infraestructura y propiedades adicionales [6]. En el esquema, cada propiedad declara su nombre lógico —el del diccionario de datos de la fuente— y, cuando difiere, el nombre físico que adopta en el almacenamiento, junto con su tipo lógico y su obligatoriedad. Este trabajo verifica las secciones de esquema y de calidad, que son las que se pueden comprobar sobre cada carga de datos.

Conviene precisar qué parte del acuerdo alcanza a expresar ese estándar. Schuiki et al. [18] descomponen un contrato de datos en cuatro aspectos —las promesas y las expectativas de cada una de las dos partes— y, en un análisis preliminar, observan que ODCS resulta adecuado para las promesas del proveedor y las expectativas del consumidor, mientras que las expectativas del proveedor y las promesas del consumidor descansan mayormente sobre campos de texto libre y quedan sin formalizar. En ese modelo el acuerdo se negocia: el consumidor evalúa las expectativas del proveedor y puede responder con una contraoferta [18]. Lo que se evalúa en este trabajo es la verificación de la parte formalizada, que es la que un proceso automático puede comprobar en cada carga.

III. Estado del arte

La verificación automática de calidad de datos cuenta con una línea de investigación consolidada. Deequ [19] introdujo una interfaz declarativa de restricciones que se traducen a consultas de agregación sobre Spark y habilitan "pruebas unitarias" para datos a escala; TFDV (*TensorFlow Data Validation*) [2] llevó la validación continua a los procesos de aprendizaje automático en producción. El trabajo de Redyuk et al. [20] es el que más se acerca a una medición de costo: evalúa la detección sobre seis clases de error sintético y compara tiempos de ejecución contra Deequ y TFDV, frente a los que su propio enfoque resulta al menos un orden de magnitud más rápido. Esa medición, sin embargo, considera el costo de la verificación de manera aislada, sin contrastarlo contra el costo del defecto que no se detecta, que es el término que decide la adopción. Estos sistemas demuestran la viabilidad técnica de la verificación declarativa, pero no cuantifican el compromiso entre ambos costos, ni lo estudian en el contexto de arquitecturas analíticas por capas.

El relevamiento de herramientas confirma una cobertura despareja de las dimensiones de calidad. Un análisis de seis herramientas de código abierto contra las dimensiones de ISO/IEC 25012 [21] muestra que la verificación de nulos tiene soporte nativo en cinco de ellas, mientras que la de integridad referencial solo en dos. Ese relevamiento no contempla los contratos de datos como objeto de estudio.

Por el lado de los contratos, la trayectoria del tema explica dónde está hoy la evidencia. Un antecedente académico temprano aparece en el contexto de los mercados de datos en la nube, donde el contrato regula los derechos de uso, la calidad, el cumplimiento normativo y el precio del dato [17], pero el concepto resurge una década después impulsado por la práctica profesional [5], con un acompañamiento académico que tanto la revisión sistemática de literatura gris de Wasser et al. [7] como el análisis conceptual de Schuiki et al. [18] describen como escaso. La adopción industrial sí está documentada: un informe de experiencia sobre cinco años de uso en manufactura detalla las ventajas observadas y las lecciones aprendidas —entre ellas, que el cambio cultural es el aspecto que más tiempo demanda implementar— [22], aunque en el registro de la experiencia y no en el de la medición controlada.

Sobre esa base empiezan a aparecer trabajos exploratorios y propuestas de sistema. Dos tesis recientes abordan la implementación de contratos: una valida que un contrato detecta corrupciones inyectadas en un conjunto plano de mil registros —nueve de once violaciones intencionales; las dos restantes requerían restricciones de enumeración que el contrato no declaraba— [23] y otra estudia el problema desde los requisitos y el ecosistema organizacional [24]. En paralelo, se propone un diseño de *lakehouse* correcto por construcción que integra contratos —entendidos como esquemas de tabla tipados—, versionado y ejecución transaccional [25], con una validación formal preliminar antes que empírica. Ninguno de estos trabajos compara estrategias de verificación entre sí, ni las sitúa en una arquitectura por capas, ni cuantifica qué cuesta verificar frente a qué cuesta no hacerlo.

Tabla I
Trabajos empíricos más cercanos según qué evalúan

Trabajo	Fallas inyectadas	Mecanismos comparados	Costo de verificar	Contrasta con el defecto
Redyuk et al. [20]	seis clases sintéticas	su enfoque, Deequ y TFDV	tiempo de ejecución	no
Papastergios y Gounaris [21]	—	funcionalidades de seis herramientas	—	no
Hasan [23]	once violaciones intencionales	un único contrato	—	no
Este trabajo	seis clases	sin verificación, motor y contrato	tiempo de ejecución	sí

La Tabla I resume qué evalúan los trabajos empíricos más cercanos.

En síntesis: la validación declarativa funciona, existe un estándar abierto para expresarla como contrato, y el tiempo que insume ejecutar una verificación ya fue medido [20]. Lo que no se midió es el compromiso. Ninguna publicación compara la verificación por contrato con las restricciones del motor sobre un mismo proceso, ni contrasta lo que cuesta verificar contra lo que cuesta reprocesar cuando el defecto no se detecta, ni sitúa esa comparación en una arquitectura por capas para determinar cuánto agrega el contrato por encima de las restricciones que el motor ya provee. Ese es el aporte de este trabajo.

IV. Desarrollo

El diseño experimental sigue los lineamientos de experimentación en ingeniería de software [26]: hipótesis enunciadas antes de la ejecución, tratamientos comparados sobre el mismo objeto, repeticiones con semilla controlada y discusión explícita de amenazas a la validez.

IV-A. Hipótesis

H1: la verificación por contratos detecta una proporción mayor de las fallas inyectadas que la ausencia de verificación. **H2**: las restricciones nativas del motor cubren las fallas estructurales pero no las semánticas (unicidad de clave de negocio, integridad referencial). **H3**: los contratos anticipan la detección a una capa más temprana, reduciendo el volumen defectuoso que alcanza el consumo. **H4**: el sobrecosto de verificar en ejecuciones sin defectos es menor que el costo de recálculo evitado a partir de cierta frecuencia de aparición de defectos.

IV-B. Proceso de referencia

Se construye un proceso de tres capas sobre el conjunto de datos público *Online Retail II* [27] (UCI Machine Learning Repository, licencia CC BY 4.0), una tabla transaccional de ventas minoristas de 1.067.371 registros. La capa de ingesta conserva el dato crudo, sin inferencia de esquema: todas las columnas se leen como texto, porque una conversión implícita en esa capa enmascararía las fallas de tipo. La capa intermedia normaliza el dato en un modelo dimensional con

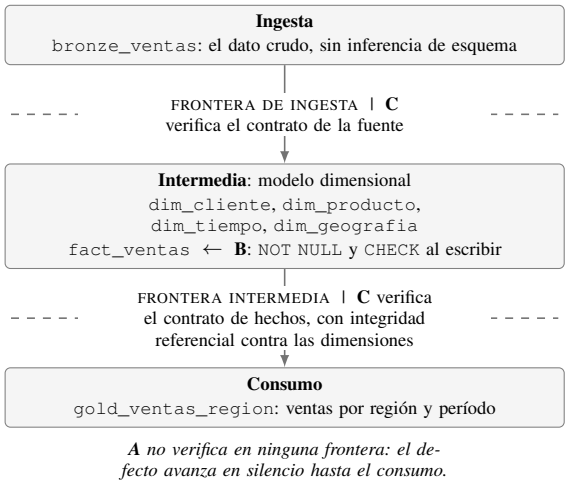


Figura 1. Proceso de referencia y punto de verificación de cada variante. La variante C verifica en las dos fronteras, antes de habilitar la capa siguiente; la variante B, dentro de la capa intermedia, al escribir la tabla de hechos.

cuatro dimensiones (cliente, producto, tiempo y geografía) y una tabla de hechos con grano de línea de factura; la capa final agrega ventas por región y período. La dimensión geográfica se resuelve contra una tabla de correspondencia estática país–región–continente construida sobre el dominio de los 43 valores de país presentes en la fuente, lo que habilita violaciones de integridad referencial genuinas cuando el origen informa un país ausente de esa correspondencia: el formato de tabla no aplica claves foráneas. La Figura 1 resume el proceso y señala en qué punto verifica cada variante.

IV-C. Población experimental

La inyección de fallas exige una línea de base sin defectos, porque de otro modo una detección no puede atribuirse a la falla inyectada. La población experimental es entonces el subconjunto de *Online Retail II* conforme al contrato de la fuente, y los criterios de descarte son las cláusulas de ese contrato y la pertenencia del país a la dimensión geográfica, no un criterio de conveniencia. El descarte se cuantifica y se informa en la Sección V. Sobre esa población de 787.144 registros y 44.863 facturas se toman muestras de al menos 50.000 registros por repetición —la mediana es de 50.013—, con semilla fija y a nivel de factura, de modo que una factura nunca se parte y el grano del modelo se preserva. Las tres variantes reciben en cada repetición exactamente la misma muestra.

IV-D. Variantes comparadas

A — Sin verificación: la lectura, defensiva en las tres variantes, deja avanzar el defecto en silencio. **B — Restricciones del motor**: la tabla de hechos se crea con restricciones `NOT NULL` sobre el identificador de cliente, la cantidad y el precio, y `CHECK` sobre sus dominios —precio no negativo y cantidad entre -1.000 y 10.000—; una violación interrumpe la escritura. Estas son las dos únicas restricciones que admite Delta Lake [16]; la plataforma Databricks permite declarar

```
schema:
- name: bronze_ventas
  properties:
  - { name: Customer ID, physicalName: Customer_ID,
      logicalType: number, required: true }
  quality:
  - { rule: unique, columns: [Invoice, StockCode] }
  - { rule: range, column: Quantity,
      min: -1000, max: 10000 }
  - { rule: range, column: Price, min: 0, max: 50000 }
```

Figura 2. Fragmento del contrato de la fuente: nombre lógico y físico del identificador de cliente y reglas de calidad que se verifican en la frontera de ingesta, en la sintaxis del verificador desarrollado.

```
schema:
- name: fact_ventas
  properties:
  - { name: country, logicalType: string, required: true }
  quality:
  - { rule: unique, columns: [invoice, stock_code] }
  - { rule: referential, column: country,
      refTable: dim_geografia, refColumn: country }
```

Figura 3. Fragmento del contrato de la tabla de hechos: dos de sus reglas de calidad, en la sintaxis del verificador desarrollado sobre la estructura de ODCS. Corresponden a las dos clases de falla que las restricciones del motor no alcanzan a cubrir; la duplicación inyectada se detiene antes, en la regla de unicidad del contrato de la fuente.

además claves primarias, foráneas y de unicidad, pero son informativas y no se verifican [28], de modo que declararlas no habría aportado detección alguna. La implementación de código abierto empleada, además, rechaza endurecer la nulabilidad de una columna ya creada [29], de modo que la restricción se declara al crear la tabla y no por alteración posterior. **C — Contratos**: especificaciones ODCS verificadas en las dos fronteras (ingesta→intermedia sobre la fuente; intermedia→consumo sobre la tabla de hechos, incluidas las reglas de integridad referencial contra las dimensiones). La comparación central es B vs. C: cuánto agrega el contrato declarativo por encima de lo que el motor ya provee.

El contrato de la fuente declara las ocho columnas del conjunto con su tipo lógico y su obligatoriedad —todas obligatorias salvo la descripción del producto—, la unicidad de la clave factura–producto y los rangos admitidos de cantidad y precio. El de la tabla de hechos declara sus columnas, la unicidad del grano de línea de factura, el rango de cantidad y las referencias del país y del cliente a sus dimensiones. El nombre físico difiere del lógico solo en el identificador de cliente, que en el diccionario de datos lleva un espacio. La Figura 2 reproduce esa declaración y las reglas de calidad del contrato de la fuente.

La Figura 3 reproduce el fragmento del contrato de la tabla de hechos que produce el resultado principal: la regla de integridad referencial, que ninguna restricción del motor puede sustituir.

Para que la diferencia de costo entre variantes sea atribuible a la verificación y no al modo de escritura, la tabla de hechos se materializa con la misma ruta en las tres: creación de la tabla vacía y posterior inserción.

Tabla II
CLASES DE FALLA, CORRUPCIÓN INYECTADA Y CLÁUSULA DEL CONTRATO QUE LA CUBRE

Clase	Corrupción en el origen	Cláusula que la cubre
Cambio de tipo	precio como texto con coma decimal	tipo numérico del precio (fuente)
Nulos en campo obligatorio	identificador de cliente vacío	obligatoriedad del cliente (fuente)
Clave de negocio duplicada	filas repetidas	unicidad factura–producto (fuente)
Deriva de esquema	columna de cantidad renombrada	columna declarada ausente (fuente)
Valor fuera de dominio	cantidad con valor centinela	rango de cantidad (fuente)
Integridad referencial	país inexistente	referencia a la dimensión geográfica (hechos)

IV-E. Clases de falla e inyección

Las seis clases se asignan a características inherentes de ISO/IEC 25012 [8], y tres de ellas —tipo, nulos y duplicados— figuran además entre las causas raíz más frecuentes de los problemas de datos relevados en procesos reales [30]; se parte de esas fuentes y no de criterio propio para evitar un sesgo favorable a alguna variante: cambio de tipo de dato (exactitud), nulos en campos obligatorios (completitud), duplicación de la clave de negocio (consistencia), deriva de esquema en el origen (consistencia), valores fuera de dominio (exactitud, en su aspecto sintáctico) y violación de integridad referencial entre hechos y dimensiones (consistencia). El generador corrompe el archivo de origen antes de la ingesta, con semilla fija y una clase por corrida. El número de clases coincide con el de Redyuk et al. [20], que inyectan seis clases sintéticas elegidas por su frecuencia en la práctica y por su uso previo en investigación sobre detección de errores; dos de las clases aquí empleadas —los nulos en campos obligatorios y el valor centinela fuera de dominio— equivalen a sus valores faltantes explícitos e implícitos. Las restantes difieren en naturaleza: aquel trabajo cubre errores de contenido —anomalías numéricas, campos permutados, erratas— mientras que las de este son violaciones estructurales y semánticas del acuerdo con la fuente, entre ellas la integridad referencial, ausente de aquel conjunto.

La Tabla II detalla la corrupción que introduce cada clase —sobre el 3 % de las filas de la muestra, salvo la deriva de esquema, que afecta la columna completa— y la cláusula del contrato que la cubre.

IV-F. Matriz y medición

La matriz comprende 6 clases $\times$ 3 variantes $\times$ 5 repeticiones, más 15 ejecuciones de control sin defectos: 105 corridas. Cada corrida se ejecuta en una sesión nueva y sobre un almacenamiento nuevo, y una ejecución inicial de calentamiento se descarta sin registrar. Se informa la mediana sobre las repeticiones.

Cada corrida sigue el mismo procedimiento: el generador escribe el archivo de origen con la falla de su clase, el proceso lo ingiere y construye las capas, y en la variante C cada frontera verifica el contrato que le corresponde antes de habilitar la capa siguiente. Una corrida se registra como detectada cuando la interrumpe un mecanismo de verificación —una violación de contrato o de restricción del motor—, junto con la capa en que ocurrió. Cualquier otro error invalida la corrida, y el análisis se niega a procesar resultados que contengan alguno, para que un fallo de infraestructura no se confunda con una detección. El volumen defectuoso se cuenta sobre la tabla de hechos antes de la verificación de la frontera intermedia, de modo que es comparable entre variantes, y el desvío del total publicado se calcula contra el total de ventas de la muestra limpia, obtenido con la misma fórmula que la capa de consumo.

La medición se toma en dos niveles. El registro de transacciones de Delta Lake (`DESCRIBE HISTORY`) constituye un instrumento externo al código desarrollado, y para las operaciones de escritura informa volumen —filas, bytes y archivos materializados— y no tiempo: la métrica de duración (`executionTimeMs`) está documentada únicamente para las operaciones de modificación, no para las de escritura [15]. Se lo emplea entonces para lo que efectivamente acredita, que es que las tres variantes materializan la misma cantidad de datos y que una diferencia de costo no proviene de escribir volúmenes distintos. El tiempo se mide por reloj, de manera separada sobre el bloque de verificación y sobre la corrida completa. La corrida completa se mide desde la lectura del archivo de origen hasta la publicación en la capa de consumo o la interrupción por detección, e incluye los conteos de volumen defectuoso y de totales publicados; el arranque de la sesión y la consulta al registro de transacciones quedan fuera de la medición. El costo se expresa en unidades relativas —tiempo de ejecución—, nunca monetarias.

Las mediciones se realizan solo en un entorno local contenedorizado con versiones fijadas —Python 3.11, OpenJDK 21, Apache Spark 3.5.3 y delta-spark 3.2.0, en modo local con dos hilos de ejecución—; un entorno de cómputo compartido y elástico no permite que un tercero reproduzca los tiempos, mientras que el contenedor sí. El código, los contratos, el detalle de la población y los resultados crudos quedan disponibles para su verificación en un repositorio público[1].

V. RESULTADOS

V-A. Caracterización de la fuente

El conteo de incumplimientos de las cláusulas del contrato de la fuente sobre el conjunto crudo, sin inyectar ninguna falla, señala 243.007 registros sin identificador de cliente (22,8 % del total), 45.947 registros que repiten la clave de negocio factura–producto, 160 valores de cantidad fuera del dominio declarado y 5 precios negativos. El descarte de los registros no conformes deja una población experimental de 787.144 registros, esto es, 26,25 % del conjunto original queda fuera. La Tabla III resume el resultado. Cada cláusula se cuenta por

[1] https://github.com/itsmeleandro/tfg-data-contracts

Tabla III
REGISTROS DEL CONJUNTO ORIGINAL QUE INCUMPLEN CADA CLÁUSULA DEL CONTRATO DE LA FUENTE

Cláusula del contrato	Registros
Identificador de cliente obligatorio	243.007
Unicidad factura–producto	45.947
Rango de cantidad	160
Rango de precio	5
Conjunto original	1.067.371
Población experimental	787.144

Tabla IV
TASA DE DETECCIÓN POR CLASE DE FALLA Y VARIANTE

Clase de falla	Dimensión	A	B	C
Cambio de tipo	Exactitud	0 %	100 %	100 %
Nulos en campo obligatorio	Completitud	0 %	100 %	100 %
Clave de negocio duplicada	Consistencia	0 %	0 %	100 %
Deriva de esquema	Consistencia	0 %	100 %	100 %
Valor fuera de dominio	Exactitud	0 %	100 %	100 %
Integridad referencial	Consistencia	0 %	0 %	100 %

separado, de modo que un registro que incumple más de una aparece en más de una fila.

V-B. *Detección por clase de falla*

La Tabla IV informa la tasa de detección por clase de falla y variante sobre las corridas con defecto inyectado. La variante A no detecta ninguna clase. La variante B detecta 67 % de las corridas y queda en cero en las clases de clave de negocio duplicada y integridad referencial. La variante C detecta 100 % de las corridas en todas las clases.

V-C. *Capa de detección*

La Figura 4 distribuye las corridas con defecto según la capa en que se produjo la detección. En la variante C, 83 % de las corridas se detiene en la capa de ingesta y 17 % en la intermedia. En la variante B, la detección ocurre íntegramente en la capa intermedia, en 67 % de las corridas, y 33 % atraviesa el proceso sin detección. En la variante A, 100 % de las corridas llega al consumo.

V-D. *Volumen defectuoso y número publicado*

La Tabla V informa, para la variante sin verificación, la cantidad de registros inyectados, cuántos alcanzan la tabla de hechos y cuánto se desvía el total de ventas publicado en la capa de consumo respecto del total calculado sobre el dato limpio. La mediana de registros defectuosos que alcanzan la tabla de hechos es de 1.500 por corrida. El desvío del total de ventas publicado alcanza 464.660,88 % en la clase de valor fuera de dominio, y es de −3,20 % y 3,08 % en las de cambio de tipo y clave de negocio duplicada, respectivamente. En la clase de nulos en campo obligatorio el desvío es de 0,00 %; en la de deriva de esquema el total no resulta calculable; y en la de integridad referencial el desvío es también de 0,00 %, aunque 1.500 líneas quedan sin región asignada en la capa de consumo. Las corridas de la variante A alcanzan la capa de consumo en 100 % de los casos y las de la variante C en 0 %.

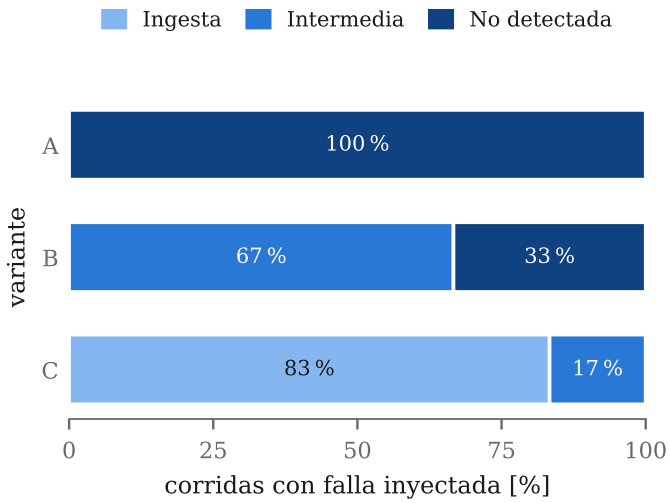


Figura 4. Capa en que cada variante detecta el defecto, sobre las corridas con falla inyectada.

Tabla V
VOLUMEN DEFECTUOSO Y DESVÍO DEL TOTAL PUBLICADO EN LA VARIANTE SIN VERIFICACIÓN (MEDIANAS)

Clase de falla	Inyectados	En hechos	Desvío del total
Cambio de tipo	1.500	1.500	−3,20 %
Nulos en campo obligatorio	1.500	1.500	0,00 %
Clave de negocio duplicada	1.500	1.500	3,08 %
Deriva de esquema	50.013	50.013	no calculable
Valor fuera de dominio	1.500	1.500	464.660,88 %
Integridad referencial	1.500	1.500	0,00 %

V-E. *Costo*

El registro de transacciones informa, para las corridas de control, una mediana de 106.191 filas y 0,84 MB materializados en las tres variantes, sin diferencia entre ellas.

En esas mismas corridas de control la mediana del tiempo de ejecución es de 9,03 s en la variante A, 11,14 s en la B y 15,34 s en la C, lo que representa un sobrecosto de 23,4 % y 70,0 %, respectivamente. La verificación de los contratos insume una mediana de 9,36 s y la declaración de las restricciones del motor, 2,14 s. Una corrida que aborta por detección cuesta 7,39 s en la variante B y 6,19 s en la C, frente a 10,85 s de una corrida completa de la variante A con falla inyectada.

Con esas medianas, el costo promedio de una corrida depende de qué proporción p de las corridas trae un defecto. Sin verificar, toda corrida cuesta 9,03 s y la que trae defecto suma el recálculo que exige descubrirlo cuando ya llegó a la capa de consumo:

$$\text{costo por corrida} = 9{,}03 + 10{,}85 \times p. \tag{1}$$

Con contratos, las corridas limpias cuestan 15,34 s y la que trae un defecto se interrumpe al detectarlo y cuesta 6,19 s:

$$\text{costo por corrida} = 15{,}34 \times (1 - p) + 6{,}19 \times p. \tag{2}$$

Las ecuaciones (1) y (2) son las rectas que unen dos puntos obtenidos de las medianas: en $p = 0$, el costo de una corrida

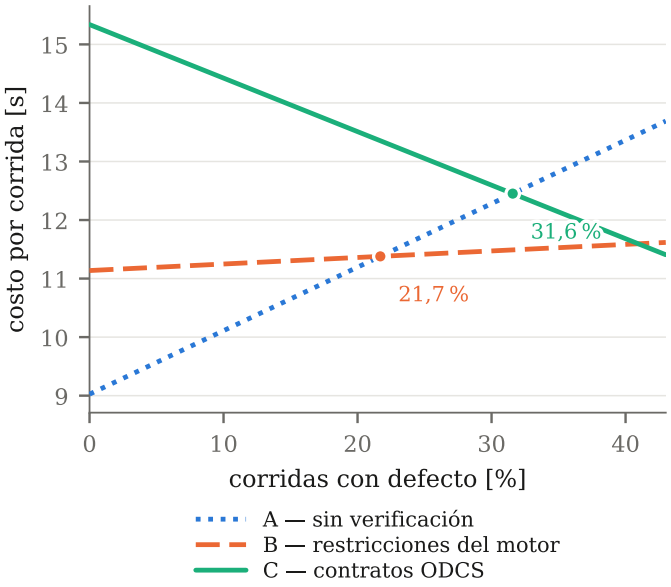


Figura 5. Costo promedio por corrida de cada variante según la proporción de corridas con defecto. Los puntos marcan el cruce con la variante sin verificación.

limpia, y en $p = 1$, el de una corrida con defecto. Las dos rectas se cruzan cuando p llega a 31,6 %: a partir de esa proporción —un defecto cada 3,2 corridas— verificar sale más barato que no hacerlo. Las restricciones del motor siguen la misma lógica, pero frenan solo 67 % de los defectos y el 33 % restante paga la corrida completa más el recálculo; su recta se cruza con la de no verificar en 21,7 %, un defecto cada 4,6 corridas. La Figura 5 traza las tres rectas en el tramo donde se producen los cruces.

VI. Discusión

VI-A. *Sobre H1: cobertura de la verificación por contratos*

La diferencia entre la variante sin verificación y la de contratos es la esperada y no admite matices: ninguna clase de falla se detecta sin verificar y todas se detectan con contratos. El resultado confirma H1 pero su interés es limitado, porque el término de comparación es la ausencia de mecanismo. El aporte está en la comparación con las restricciones que el motor ya provee.

VI-B. *Sobre H2: el límite de las restricciones del motor*

Las restricciones nativas cubren las fallas estructurales —nulos y valor fuera de dominio— y también el cambio de tipo y la deriva de esquema, aunque por vía indirecta: el precio con coma decimal no convierte a número y la columna renombrada desaparece; en ambos casos la lectura defensiva deja nulos y es la restricción de nulabilidad la que finalmente aborta la escritura. El motor detecta la consecuencia, no la causa, y el mensaje de error apunta a una columna nula y no a un cambio en el origen.

Las clases de clave de negocio duplicada y integridad referencial quedan sin detectar, lo que confirma H2. La razón es estructural y no una limitación de configuración. Delta Lake no implementa claves primarias, foráneas ni de unicidad [16], y la plataforma Databricks, que admite declararlas, las trata como informativas: pueden mejorar el rendimiento de las consultas, pero no rechazan una escritura que las viole [28]. Es decir que, aun donde el mecanismo forma parte del vocabulario del motor, no forma parte de su comportamiento, distinción que importa porque un modelo dimensional heredado de la práctica del *data warehouse* supone justamente lo contrario, que la clave foránea se aplica.

Este resultado coincide con lo que muestra el relevamiento de herramientas sobre la cobertura despareja de las dimensiones de calidad y, en particular, sobre la integridad referencial, que solo dos de las seis herramientas relevadas ofrecen como verificación nativa [21], y se verifica también en el propio estándar empleado: la biblioteca de métricas de calidad que incorporó en su versión 3.1 cubre nulos, valores faltantes, valores inválidos, duplicados —incluida la clave compuesta— y recuento de filas, pero no incluye una métrica de integridad referencial; el estándar permite declarar la clave foránea como relación del esquema y deja su verificación a cargo de la implementación [6]. La verificación por contrato cubre entonces esa clase no porque el estándar la verifique, sino porque el verificador desarrollado la resuelve como una consulta sobre las dimensiones, sin depender de que el motor la implemente.

La cobertura de un contrato depende, a su vez, de lo que declara: en el trabajo de Hasan, las dos violaciones que el contrato no detectó correspondían a restricciones de enumeración que no estaban declaradas [23]. La detección total de la variante C se explica del mismo modo, porque cada clase inyectada tiene una cláusula que la cubre (Tabla II).

VI-C. *Sobre H3: la capa de detección*

Los contratos anticipan la detección a la frontera de ingesta en todas las clases salvo la de integridad referencial, que por naturaleza requiere que las dimensiones ya estén construidas y se detecta por eso en la frontera intermedia. La consecuencia práctica es que, en esas clases, el defecto no llega a materializarse en el modelo dimensional; en la variante sin verificación, en cambio, una mediana de 1.500 registros defectuosos alcanza la tabla de hechos y el proceso completa la publicación en la capa de consumo. H3 queda confirmada.

El dato más relevante de la Tabla V no es el volumen sino el desvío del número publicado, porque es lo que efectivamente ve quien toma una decisión. La clase de valor fuera de dominio desvía el total publicado en 464.660,88 % con solo 1.500 registros defectuosos, y la deriva de esquema lo vuelve directamente no calculable. En el otro extremo, la clase de nulos en campo obligatorio no altera el total en absoluto: el defecto no se manifiesta en la medida agregada sino en la imposibilidad de atribuir esas ventas a un cliente, lo que muestra que el volumen defectuoso y el impacto en el consumo son dos magnitudes distintas y que ninguna de las dos alcanza por separado para describir el daño.

VI-D. Sobre H4: el compromiso de costo

H4 no se responde con un valor único sino con la curva de la Figura 5, y el sentido de la respuesta depende de la frecuencia con que aparezcan los defectos. Verificar tiene un costo fijo que se paga en toda ejecución, incluidas las limpias, mientras que el ahorro se materializa solo cuando hay defecto. Con los costos medidos, la variante de contratos cuesta menos que la de no verificar a partir de 31,6 % de corridas con defecto, y las restricciones del motor a partir de 21,7 %.

El tiempo que insume la verificación no se traslada al total de manera aditiva: las consultas del contrato fuerzan lecturas que la variante sin verificación realiza igualmente, solo que más adelante en el proceso, de modo que el sobrecosto neto en corridas limpias (6,31 s) resulta menor que el tiempo atribuido al bloque de verificación (9,36 s). Medir el costo de verificar aislando ese bloque sobreestima entonces su impacto, y es la diferencia de extremo a extremo la que corresponde llevar al modelo.

Esa lectura es conservadora por construcción, y conviene ser explícito al respecto: el cálculo cuenta como costo del defecto no detectado únicamente el recálculo del proceso, que es la parte medible. No incluye el tiempo de diagnóstico, ni el de la corrección en el origen, ni las decisiones que se tomaron sobre el número equivocado mientras el defecto estuvo publicado. Cuanto mayor es ese costo, más empinada es la recta de no verificar y antes se cruza con la de verificar.

Hay dos razones para considerar que el costo realista del recálculo está lejos de una sola corrida. La primera es interna: la Tabla V muestra un desvío de varios órdenes de magnitud en el número publicado, y esa magnitud no entra en el cálculo. La segunda proviene de la literatura profesional, que debe leerse con las reservas propias de la literatura gris [3]: una encuesta a doscientos profesionales de datos informa un promedio de quince horas para resolver cada incidente de calidad y, sobre todo, que tres de cada cuatro encuestados señalan que, siempre o casi siempre, quien detecta primero el problema es el usuario de negocio que consume el dato [4]. Un costo de descubrimiento y corrección medido en horas frente a una corrida medida en segundos ubica ese costo varios órdenes por encima de una corrida, lo que traslada el punto de equilibrio a frecuencias de defecto muy inferiores a la del cálculo base: si el recálculo costara cinco corridas limpias, a los contratos les alcanzaría un defecto cada 8,6 corridas, y uno cada 15,7 si costara diez.

VI-E. Amenazas a la validez

De conclusión: la detección resultó idéntica en las 5 repeticiones de cada combinación de clase y variante —cada celda de la Tabla IV corresponde a una detección nula o total—, de modo que la conclusión sobre la cobertura no depende del número de repeticiones. Para el costo, la mediana sobre 5 repeticiones atenúa el efecto de valores extremos, pero no alcanza para estimar intervalos de confianza estrechos; por eso las conclusiones de costo se apoyan en diferencias de extremo a extremo y no en el tiempo aislado de un bloque.

Interna: la variabilidad de la máquina se mitiga con medianas sobre 5 repeticiones, con una corrida de calentamiento descartada y con una sesión nueva por corrida; las tres variantes reciben la misma muestra en cada repetición. El sesgo en el diseño de fallas se mitiga apoyándolas en taxonomías publicadas, y los contratos se especificaron antes que el generador. La ruta de escritura de la tabla de hechos es común a las tres variantes y el registro de transacciones confirma que las tres materializan el mismo volumen, de modo que la diferencia de costo no proviene de escribir cantidades distintas de datos. Un residuo inevitable es que el costo de verificación se mide por reloj y no con ese registro, que solo instrumenta el volumen de las escrituras.

Externa: un único conjunto de datos, un volumen acotado, un único motor de procesamiento y un solo motor de almacenamiento; la generalización se enuncia como línea futura. La frecuencia de defectos no se observa empíricamente sino que se parametriza, razón por la cual H4 se responde con una curva y no con un valor. La población experimental excluye los registros no conformes del conjunto original, lo que la vuelve más limpia que la fuente real; esa decisión es necesaria para atribuir las detecciones a las fallas inyectadas, pero implica que las tasas medidas no son tasas de detección sobre datos de producción.

De constructo: el costo se mide en tiempo, no en moneda, y el volumen materializado se usa solo como control; la detección se registra de manera binaria y no evalúa la utilidad diagnóstica del mensaje de error, aunque el caso de la deriva de esquema en la variante B muestra que esa utilidad no es igual en todos los mecanismos.

VII. Conclusiones

La verificación declarativa por contrato cubre clases de defecto que las restricciones del motor de almacenamiento no pueden cubrir, y la razón no es de madurez de las herramientas sino de diseño: un formato de tabla que no implementa unicidad ni claves foráneas deja fuera de alcance precisamente los defectos de naturaleza semántica, que son los que atraviesan un proceso por capas sin producir error. Expresar esas expectativas como consultas declarativas sobre el propio modelo, y no como restricciones del motor, es lo que las vuelve verificables.

El aporte de anticipar la detección no se agota en detectar más. Ubicar la verificación en la frontera de ingesta evita que el defecto se materialice en el modelo dimensional, y con eso evita que un número incorrecto quede publicado y disponible para decidir. Los dos efectos —volumen defectuoso y desvío del número publicado— son independientes entre sí: hay defectos que no alteran una medida agregada y otros que, con el mismo volumen, la distorsionan por completo. Evaluar un mecanismo de detección por una sola de esas magnitudes conduce a conclusiones equivocadas.

La decisión de adoptar contratos no admite una respuesta uniforme, porque verificar cuesta en cada ejecución y el ahorro se materializa solo cuando el defecto aparece. La respuesta es una condición sobre la frecuencia esperada de defectos y sobre

el costo de descubrirlos tarde: cuanto más caro sea corregir un defecto después de publicado, menos defectos hacen falta para justificar la verificación. Para un equipo que recibe datos de un origen que no controla, ese es el criterio con el que conviene tomar la decisión, y este trabajo aporta el método para estimarlo con mediciones propias.

Lo que no queda demostrado es que estas magnitudes se trasladen a otras escalas, motores o dominios: la medición es de un conjunto de datos, un volumen y un motor. Tampoco queda evaluado el costo organizacional de acordar y mantener un contrato entre equipos, que la experiencia industrial reseñada señala como el aspecto que más tiempo demanda y que ninguna medición técnica captura.

VIII. Líneas futuras

Quedan abiertas la repetición del experimento sobre el conjunto de datos completo, sin descartar los registros que incumplen el contrato de la fuente, para estimar la detección sobre datos de producción; la medición directa del costo de descubrir y corregir un defecto que ya llegó al consumo, que este trabajo trata como parámetro, mediante la simulación de ese ciclo; la replicación sobre múltiples conjuntos de datos, volúmenes y plataformas, incluidas las administradas; la evaluación en procesamiento continuo —aspecto que el relevamiento de herramientas señala como desatendido [21]— ; la medición de la utilidad diagnóstica de los mensajes de error; la negociación organizacional de los contratos entre equipos productores y consumidores [18], [22]; la verificación de los aspectos que el estándar todavía deja en texto libre —las expectativas del proveedor y las promesas del consumidor— [18]; la migración de las reglas de calidad a la sintaxis de métricas y relaciones que el estándar incorporó en su versión 3.1 [6], para verificarlas con implementaciones de referencia; y la generación asistida de contratos a partir de metadatos existentes, incluida la extracción automática de linaje.

Referencias

[1] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, vol. 28, 2015, pp. 2503–2511. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2015/hash/86df7dcfd896fcaf2674f757a2463eba-Abstract.html

[2] E. Breck, N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich, "Data validation for machine learning," in *Proc. Mach. Learn. Syst. (MLSys)*, vol. 1, 2019, pp. 334–347. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2019/hash/928f1160e52192e3e0017fb63ab65391-Abstract.html

[3] V. Garousi, M. Felderer, and M. V. Mäntylä, "Guidelines for including grey literature and conducting multivocal literature reviews in software engineering," *Information and Software Technology*, vol. 106, pp. 101–121, Feb. 2019. [Online]. Available: https://doi.org/10.1016/j.infsof.2018.09.006

[4] M. Segner, "The annual state of data quality survey," Monte Carlo (blog), May 2023, Accessed: Oct. 21, 2026. [Online]. Available: https://montecarlo.ai/blog-data-quality-survey

[5] A. Jones, *Driving Data Quality with Data Contracts*. Packt Publishing, 2023.

[6] Bitol, *Open Data Contract Standard (ODCS), version 3.1.0*, LF AI & Data Foundation, Dec. 2025, Accessed: Oct. 21, 2026. [Online]. Available: https://bitol-io.github.io/open-data-contract-standard/v3.1.0/

[7] J. Wasser, I. Kumara, G. Monsieur, W.-J. van den Heuvel, and D. A. Tamburri, "Data contracts in data mesh: A systematic gray literature review," in *Proc. 15th Int. Symp. Bus. Model. Softw. Des. (BMSD)*, ser. Lecture Notes in Business Information Processing, vol. 559. Cham, Switzerland: Springer, 2026, pp. 21–38. [Online]. Available: https://doi.org/10.1007/978-3-031-98033-6_2

[8] *Software engineering — Software product Quality Requirements and Evaluation (SQuaRE) — Data quality model*, ISO/IEC Std. 25012:2008, 2008. [Online]. Available: https://www.iso.org/standard/35736.html

[9] R. Y. Wang and D. M. Strong, "Beyond accuracy: What data quality means to data consumers," *Journal of Management Information Systems*, vol. 12, no. 4, pp. 5–33, 1996. [Online]. Available: https://doi.org/10.1080/07421222.1996.11518099

[10] C. Batini, C. Cappiello, C. Francalanci, and A. Maurino, "Methodologies for data quality assessment and improvement," *ACM Computing Surveys*, vol. 41, no. 3, 2009, art. no. 16. [Online]. Available: https://doi.org/10.1145/1541880.1541883

[11] M. Armbrust, A. Ghodsi, R. Xin, and M. Zaharia, "Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics," in *Proc. 11th Annu. Conf. Innovative Data Syst. Res. (CIDR)*, Jan. 2021. [Online]. Available: https://www.cidrdb.org/cidr2021/papers/cidr2021_paper17.pdf

[12] A. A. Harby and F. Zulkernine, "From data warehouse to lakehouse: A comparative review," in *Proc. IEEE Int. Conf. Big Data (Big Data)*, Osaka, Japan, Dec. 2022, pp. 389–395. [Online]. Available: https://doi.org/10.1109/BigData55660.2022.10020719

[13] Databricks, "What is the medallion lakehouse architecture?" Databricks Documentation, 2026, Accessed: Oct. 21, 2026. [Online]. Available: https://docs.databricks.com/aws/en/lakehouse/medallion

[14] M. Armbrust, T. Das, L. Sun, B. Yavuz, S. Zhu, M. Murthy, J. Torres, H. van Hovell, A. Ionescu, A. Łuszczak, M. Świtakowski, M. Szafrański, X. Li, T. Ueshin, M. Mokhtar, P. Boncz, A. Ghodsi, S. Paranjpye, P. Senster, R. Xin, and M. Zaharia, "Delta Lake: High-performance ACID table storage over cloud object stores," *Proceedings of the VLDB Endowment*, vol. 13, no. 12, pp. 3411–3424, Aug. 2020. [Online]. Available: https://doi.org/10.14778/3415478.3415560

[15] Databricks, "Table history schema and operation metrics," Databricks Documentation, 2026, Accessed: Oct. 21, 2026. [Online]. Available: https://docs.databricks.com/aws/en/tables/history-schema

[16] Delta Lake, "Constraints," Delta Lake Documentation, 2026, Accessed: Oct. 21, 2026. [Online]. Available: https://docs.delta.io/delta-constraints/

[17] H.-L. Truong, G. R. Gangadharan, M. Comerio, S. Dustdar, and F. De Paoli, "On analyzing and developing data contracts in cloud-based data marketplaces," in *Proc. IEEE Asia-Pacific Services Comput. Conf. (APSCC)*, Jeju, South Korea, Dec. 2011, pp. 174–181. [Online]. Available: https://doi.org/10.1109/APSCC.2011.65

[18] L. Schuiki, A. Wider, and S. Harrer, "Data contracts: Structuring promises and expectations in data exchange," in *Proc. IEEE Int. Conf. Web Services (ICWS)*, Sydney, Australia, Jul. 2026, pp. 659–661. [Online]. Available: https://doi.org/10.1109/ICWS72778.2026.00084

[19] S. Schelter, D. Lange, P. Schmidt, M. Celikel, F. Biessmann, and A. Grafberger, "Automating large-scale data quality verification," *Proceedings of the VLDB Endowment*, vol. 11, no. 12, pp. 1781–1794, 2018. [Online]. Available: https://doi.org/10.14778/3229863.3229867

[20] S. Redyuk, Z. Kaoudi, V. Markl, and S. Schelter, "Automating data quality validation for dynamic data ingestion," in *Proc. 24th Int. Conf. Extending Database Technol. (EDBT)*, 2021, pp. 61–72. [Online]. Available: https://doi.org/10.5441/002/edbt.2021.07

[21] V. Papastergios and A. Gounaris, "A survey of open-source data quality tools: Shedding light on the materialization of data quality dimensions in practice," 2024, arXiv:2407.18649. [Online]. Available: https://arxiv.org/abs/2407.18649

[22] S. Oppold, M. Fritz, and L. Woltmann, "Data contracts to leverage (de-)centralized data management in manufacturing industries: An experience report," in *Proc. Datenbanksysteme für Business, Technologie und Web (BTW)*, ser. Lecture Notes in Informatics (LNI), vol. P-361. Bonn, Germany: Gesellschaft für Informatik, 2025, pp. 731–743. [Online]. Available: https://doi.org/10.18420/BTW2025-39

[23] M. Hasan, "Data contracts: Implementation and validation," Master's thesis, Åbo Akademi Univ., Vaasa, Finland, 2026. [Online]. Available: https://www.doria.fi/handle/10024/195332

[24] E. Svensson and M. Torsby, "Using data contracts to fulfill requirements on data in an intra-organizational data ecosystem," Master's thesis, Dept. Comput. Sci., Lund Univ., Lund, Sweden, 2025, LU-CS-EX:

2025-15. [Online]. Available: https://lup.lub.lu.se/student-papers/search/publication/9200993
[25] W. Sheng, J. Wang, M. Barros, A. Montana, J. Tagliabue, and L. Bigon, “Building a correct-by-design lakehouse: Data contracts, versioning, and transactional pipelines for humans and agents,” Mar. 2026, arXiv:2602.02335v3 [cs.DC]. [Online]. Available: https://arxiv.org/abs/2602.02335v3
[26] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Berlin, Heidelberg, Germany: Springer, 2012. [Online]. Available: https://doi.org/10.1007/978-3-642-29044-2
[27] D. Chen, “Online Retail II,” UCI Machine Learning Repository, 2012, Accessed: Oct. 21, 2026. [Online]. Available: https://doi.org/10.24432/C5CG6D
[28] Databricks, “Constraints on Databricks,” Databricks Documentation, 2026, Accessed: Oct. 21, 2026. [Online]. Available: https://docs.databricks.com/aws/en/tables/constraints
[29] scottsand-db, “Cannot use `SET NOT NULL` constraint,” GitHub issue #831, delta-io/delta, Nov. 2021, Accessed: Oct. 21, 2026. [Online]. Available: https://github.com/delta-io/delta/issues/831
[30] H. Foidl, V. Golendukhina, R. Ramler, and M. Felderer, “Data pipeline quality: Influencing factors, root causes of data-related issues, and processing problem areas for developers,” *Journal of Systems and Software*, vol. 207, 2024, art. no. 111855. [Online]. Available: https://doi.org/10.1016/j.jss.2023.111855

BIOGRAFÍA

Leandro Ismael Rojas es estudiante de la Licenciatura en Informática de la Universidad de Palermo, Buenos Aires, Argentina. Se desempeña como ingeniero de datos desde 2021, especializado en plataformas analíticas sobre Databricks y Microsoft Azure, y cuenta con las certificaciones Databricks Certified Data Engineer Associate y Professional. Sus áreas de interés incluyen la calidad de datos, los contratos de datos y las arquitecturas analíticas por capas.